

CHAPTER 19

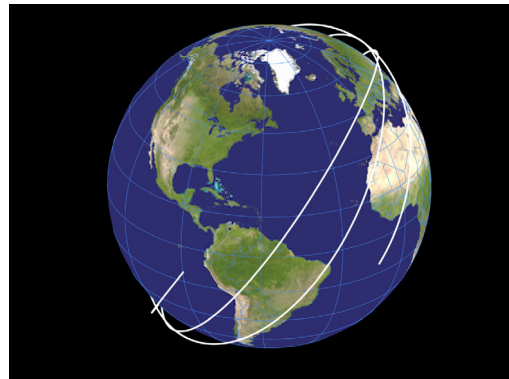
Passive control-system design

19.1. Introduction

Many satellites do not require tight pointing. In some spacecraft, passive control systems can meet the pointing requirements. This chapter discusses a design that uses a torque rod and magnetoresistive dampers.

A very important point is that magnetoresistive dampers do not work without the torque rod. The spacecraft will align the torque rod with the magnetic-field vector. The dampers then damp rates about that orientation. The torque rod only assures alignment with the magnetic-field vector. We can design the spacecraft to have a preferred orientation via the gravity gradient to get a desired Earth-pointing attitude.

```
1 %% Compute the ISSOrbit
2
3 [e1, jD0] = ISSOrbit;
4 p         = Period(e1(1));
5 t         = linspace(0,3*p,1000);
6 r         = RVOrbGen(e1,t);
7
8 PlotOrbit(r,t,jD0)
```



Example 19.1: ISS orbit in the Earth-fixed frame.

19.2. ISS orbit

The CubeSat will separate from the International Space Station (ISS). The ISS orbit is given in Table 19.1.

The corresponding two-line elements are

```
ISS
1 25544U 98067A   20092.51597385   .00016717   00000-0   10270-3   0   9025
2 25544   51.6440   7.8556 0005107   74.2499 285.9214 15.48942018 20129
```

The two-line elements are defined in Table 19.2. The decimal point is assumed in some of the terms.

Two-line elements change with time so it is best to use the latest.

Table 19.1 ISS orbit.

Element	Value
Semimajor axis	6799.7 (km)
Inclination	51.7 (deg)
Ascending Node	7.9 (deg)
Argument of Perigee	78.1 (deg)
Eccentricity	0.0014
Mean Anomaly	-77.9 (deg)
Epoch	2 458 941.0 (days)

Table 19.2 Two-line elements.

Line	Element	Description
1	1	Satellite number
1	2	Classification (The U for unclassified)
1	3	International Designator (Last two digits of launch year)
1	4	International Designator (Launch number of the year)
1	5	International Designator (Piece of the launch)
1	6	Epoch Year (Last two digits of year)
1	8	Epoch Day (Fractional Day of year)
1	9	First Time Derivative of Mean Motion divided by six
1	10	Second Time Derivative of Mean Motion divided by six
1	11	BStar drag term
1	12	The number 0
1	13	Element number
1	14	Checksum
2	1	Satellite number
2	2	Inclination (deg)
2	3	Right ascension of the ascending node
2	4	Eccentricity
2	5	Argument of perigee (deg)
2	6	Mean anomaly (deg)
2	7	Mean motion (revs/day)
2	8	Revolution number at epoch (revs)
2	9	Checksum

19.3. Gravity gradient

Our spacecraft will be a 1U CubeSat. The CubeSat model is shown in Fig. 19.1. This model is generated using the GGCubeSatModel.m script, that uses MATLAB® CAD modeling functions from the Spacecraft Control Toolbox. The primary purpose of this CAD model is to create the geometric configuration of the vehicle to support disturbance calculations and for computing the mass properties. It helps us to locate the

boom position to produce the desired gravity-gradient stability. The CAD tools are also helpful in laying out the spacecraft components.

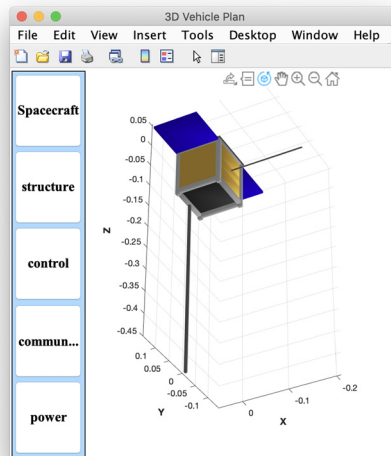


Figure 19.1 CubeSat model.

The script can also be run with the antenna, boom, and panels undeployed as shown in Fig. 19.2.

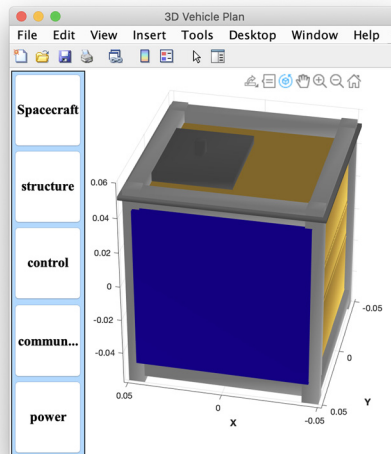


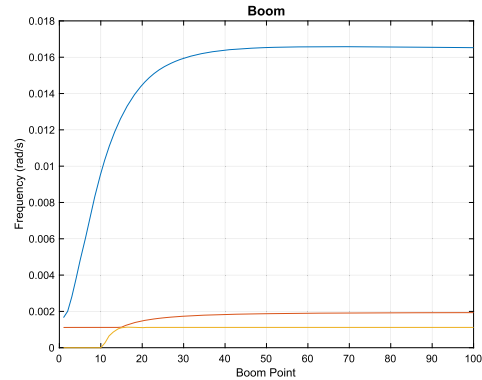
Figure 19.2 CubeSat prior to deployment. The box at the top is the folded boom with the tip mass.

The boom is offset in x to get the required libration frequencies. It also makes it easier to fold up for launch. The frequencies are shown in Example 19.2 as a function of boom length. Beyond that, there is no advantage to a longer boom. The magnetometer is also located on the boom to keep it away from the magnetic fields on the spacecraft.

```

1 %% Determine the libration frequencies
2
3 el = ISSOrbit;
4
5 inr = [0.0013      0      -0.0002;...
6        0      0.0013      -0.0000;...
7        -0.0002      -0.0000      0.0006];
8
9 m = 100;
10 r = [0.45*ones(1,m); zeros(1,m); linspace(0,4.5,m)
11      ];
12 n = 2*pi/Period(el(1));
13 GravityGradientBoom( inr , n , r , 0.2 )

```



Example 19.2: Libration frequencies.

The CubeSat attitude control components are shown in Fig. 19.3. The four magnetic-hysteresis dampers are aligned along the xy diagonals on the inside of the bottom face shown, which is the $-z$ face of the body. The torque rod is aligned along the z -axis, and the magnetometer is mounted on the inside of the $+x$ face of the body.

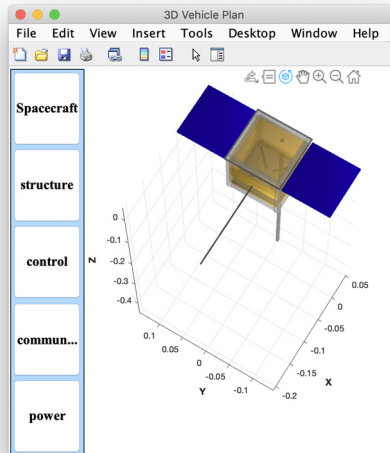


Figure 19.3 CubeSat attitude control components.

19.4. Simulations

There are two cases of interest;

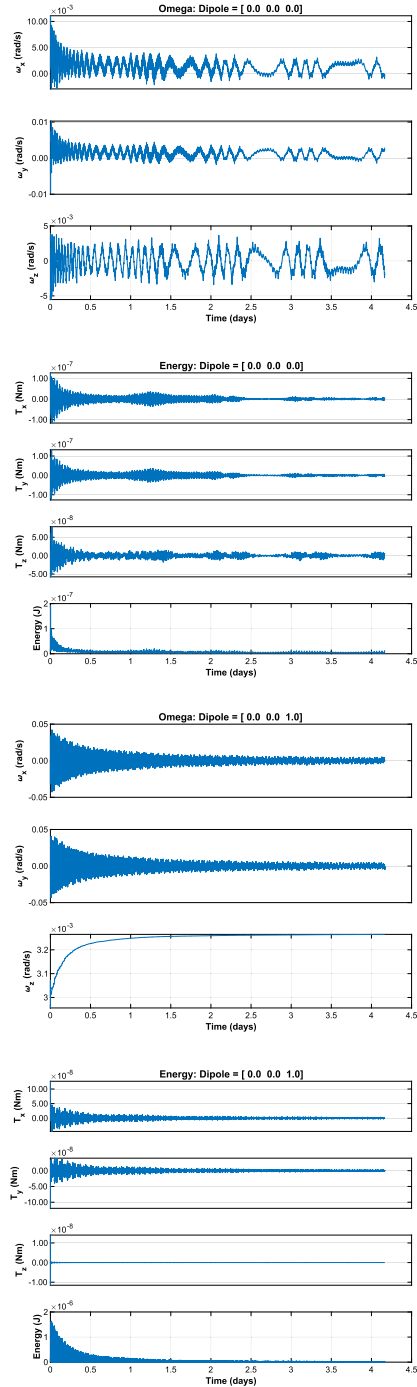
1. With the torque rod;
2. Without the torque rod.

In both examples, the spacecraft will start with an angular rate. Example [19.3](#) shows both. Without the dipole energy, the angular rate does not converge. The angular rate converges to a steady rate with the dipole, and energy is efficiently dissipated.

```

1 %% Constants
2 secInDay = 86400;
3 mu0 = 4e-7*pi;
4
5 %% User inputs
6 dateStart = [2023 6 1 0 0 0];
7 tDuration = 100*3600;
8 q0 = [1;0;0;0];
9 dT = 2;
10 omega0 = [0.001; -0.0002; 0.003];
11 d = RHSRigidBodyWithDamping;
12 d.dipole = [0;0;1];
13 [d.inertia, d.mass] = InertiaCubeSat('IU');
14 cDamp = 0.0001;
15
16 %% Start date
17 jDStart = Date2JD(dateStart);
18
19 % Orbit
20 [el,jD0] = ISSOrbit;
21 el(6) = el(6) + (jDStart-jD0)*secInDay*2*pi /
    Period(el(1));
22 [r,v] = El2RV(el);
23
24 % Time vector
25 n = ceil(tDuration/dT);
26 t = linspace(0,tDuration,n);
27 jD = jDStart + t/secInDay;
28
29 d.dampingType = 0;
30 d.dampingData.Br = 0.004;
31 d.dampingData.Bs = 0.025;
32 d.dampingData.Hc = 12;
33 % Damper rod unit vectors
34 d.dampingData.u = [0 0 0 1 1 1; 1 1 1 0 0 0; 0 0 0 0 0 0];
35 % Dimensions are radius 1 mm by 95 mm
36 d.dampingData.v = pi*0.001^2*0.095*ones(1,size(d.
    dampingData.u,2));
37 d.dampingFun = @RSHHysteresisDamper;
38 uECI = QTForm(q0,d.dampingData,u);
39 [bI,bIDot] = BDipole(r,jDStart,v);
40 hMag = Dot(uECI,bI)/mu0;
41 hMagDot = Dot(uECI,bIDot)/mu0;
42 z = BFromHHysteresis(hMag,hMagDot,d.
    dampingData);
43 x = [r;v;q0;omega0;z];
44 xP = zeros(20,n);
45
46 %% Simulation loop
47 for k = 1:n
48 % Plotting
49 [~,p] = RHSRigidBodyWithDamping(x,t(k),d);
50 omega = x(11:13);
51 energy = 0.5*omega'*d.inertia*omega;
52 xP(:,k) = [x(7:end);p.torqueDipole;p.
    torqueDamper;energy];
53 % Integrate
54 x = RK4(@RHSRigidBodyWithDamping,x,dT,t(k),d);
55 end
56
57 %% Plotting
58 yL = [d.states(:)' {'M_x_u(ATM^2)'} {'M_y_u(ATM^2)'}
    {'M_z_u(ATM^2)'} ...
    {'T_x_u(Nm)'} {'T_y_u(Nm)'} {'T_z_u(Nm)'} {'Energy_u(
    J)'}];
59
60
61 s = sprintf('Omega: uDipole_u=[%4.1f,%4.1f,%4.1f]',
    d.dipole);
62 TimeHistory(t,xP(5:7,:),yL(11:13),s);
63 s = sprintf('Energy: uDipole_u=[%4.1f,%4.1f,%4.1f]',
    d.dipole);
64 TimeHistory(t,xP(17:20,:),yL(23:26),s);

```



Example 19.3: Energy dissipation. The first two plots show the behavior without a dipole.